

Lab 4: Advanced SQL 2

In this lab you will get some extra practice with complex joins and aggregates, and you will explore more advanced SQL queries that involve *nesting*.

Objectives:

- Practice queries that involve joins and aggregates
- Identify when special joins (outer joins, self joins) are needed.
- Write uncorrelated nested queries
- Write a nested query where the outer and inner queries are correlated.
- Practice on real-world data.

Preparation:

You will be working with the ***project_sales*** database and in the last part with the ***imdb*** database. Connect to each database with the `\c` command in `psql`.

Comments on developing more complex queries: As with writing code in a programming language, it helps to develop your queries incrementally. This is especially true for complex queries. The practice database is simple, so it is easy to check your results, but this is more difficult in real-world databases. Try executing the join first and check that it looks correct before adding more constraints.

You can also execute smaller queries to explore parts of the database that your final query will use. You can also execute a `select *` query to see all of the columns in a join and then you will have a better idea of exactly what columns you need in your final select clause. You can start with a simpler query and then add constraints as you check and develop the final query.

Another technique is to execute a probing query with one specific “hard-coded” constraint, such as a specific name that is in the data set you are looking for. If you get the result you are looking for for that specific name, then you can generalize the query.

For the best learning experience: run your query against the database on your machine while developing it. Do not use gradescope to develop and debug your query until you are sure it is correct. Explore the database and use the techniques above to check that your query is correct. That way you will (1) get to “know” the database, and (2) develop good querying skills for when you are in an environment where there is no gradescope evaluator.

Remember that you can see the structure of the database by using the \d and \dt commands. You can also check on the primary and foreign key constraints (by using the \d and \dt commands) as they implement relationships between tables.

Let us know if you have tried to get a query to work but got stuck or have any questions!

Part 1: Practicing joins and aggregates

These queries are on the *project_sales* database.

Q1: Write a query that returns the employee id, name, and phone for employees who are not assigned to a project. Submit the query in file Q1.sql.

Q2: For each company, list the name of the company in the first column, and the number of products it produces in the second column. Make sure all companies appear in the result. Submit the query in file Q2.sql.

Q3: For each employee, list the employee’s name as the first column, and the number of people they manage in the second column. Submit the query in file Q3.sql.

Q4: Generate a sales report that shows, for each month for which there have been sales, how many different categories of products were sold that month. *Hint: pay attention to not overcount. If two products of the same category were sold in the month of June, this category should only be counted once.* Your output should return the name of the month in the first column, and the number of categories in the second column. In your practice dataset, the result will have 6 rows. Submit the query in file Q4.sql.

Q5: In the previous query, the result does not list all months of the year, but only those months for which there was some sale. Modify your previous query, so that the result lists all months of the year (so you will have 12 rows in the result); the corresponding category count should obviously be 0 for months for which there were no sales.

The challenge in this query is that you don't have all the months listed in some relation; so, how can the database get that information? You could provide it as part of your query! Look at the following example:

```
WITH months (name) AS (  
    values  
    ('January'),  
    ('February'),  
    ('March'),  
    ('April'),  
    ('May'),  
    ('June'),  
    ('July'),  
    ('August'),  
    ('September'),  
    ('October'),  
    ('November'),  
    ('December'))  
SELECT *  
FROM months;
```

The `WITH` clause (also called a CTE—Common Table Expression) defines a temporary result set that you can reference within your query. In this example, the database creates the temporary relation `months`, which is used in the query that follows. Note that this relation does not persist in your database; if you execute the query “`SELECT * FROM months`” separately, Postgres will tell you that this relation does not exist.

Use your new knowledge about the `WITH` clause to write a query that returns all months of the year, and for each month the number of different categories of products sold. Submit the query in file `Q5.sql`.

Q6: Write a query that returns the total revenue from sales in each month. Like in Q5, return all months of the year; the first column should list the name of the month, and the second column should report the revenue. E.g., 2 Gizmos were sold in February, the revenue for that sale would be $2 \times \$19.99 = \39.98 . *Hint: To get the correct total to show up for month with no sales, look up the function [COALESCE](#).*
Submit the query in file Q6.sql.

Part 2: Nested Queries

The output of a query is a relation, and that relation can be used inside another query. Thus, queries can be “nested”. See for example the following query:

```
SELECT pname
FROM Product
WHERE price = (SELECT min(price) FROM Product);
```

The inner query (`SELECT min(price) FROM Product`) computes the overall smallest price in your dataset, and the outer query (`SELECT pname FROM Product WHERE price = ...`) returns those products that have that minimum price. Thus, this overall query finds the cheapest products.

You can have several levels of nesting. However, note that nesting can make your queries more complex in some cases, and less efficient.

The example above is an uncorrelated nested query. This means that the inner query can execute independently of the outer query. In a correlated query, a column in the outer query is also used in the inner query. The inner query is executed once for every row of the outer query. Contrast with the following example:

```
SELECT p1.pname
FROM Product p1
WHERE p1.price IS NOT NULL
      AND NOT EXISTS (SELECT *
                      FROM Product p2
                      WHERE p2.price < p1.price);
```

The query above also computes the cheapest products, but does so with a correlated subquery (note that p1 is referenced inside the subquery). EXISTS is a special keyword that evaluates to TRUE when the result of the inner query is not empty. Consequently, NOT EXISTS evaluates to TRUE when the result of the inner query is empty.

```
SELECT pname
FROM Product
WHERE price <= ALL(SELECT price
                   FROM Product
                   WHERE price IS NOT NULL);
```

The above is yet another way to write the same query. The predicate using the keyword ALL evaluates to TRUE if it is satisfied for all values in the subquery. Predicates using the keyword ANY evaluate to TRUE if they are satisfied for at least one value in the subquery.

Write the following nested queries:

Q7: Write a query to find the country that produces the cheapest product. Use a nested query. Return one column: the country name. *Hint: write an inner query on the product table to get the cheapest (lowest price) product, then use an outer query to find the country using the product and company tables and incorporating the inner query in a where clause.* Submit the query in file Q7.sql.

Q8: Write a nested query that returns two columns: pname and month. For each product, find the month during which the product had the most sales out of the year. For example, if out of all the months of the year, Multitouch had the most sales in December, then the tuple (Multitouch, December) should be in the result. Return the product name and month. Note: in case of a tie, both months should appear in the result set. You should have 9 rows.

Hint: write this as a correlated query using the sales table in both inner and outer queries, correlating on the product name.

Submit the query in file Q8.sql.

Q9: Write a nested query to return the name of the company that makes the most products. *Hint: use grouping clauses, and remember that a select query returns a table, which you can assign to an alias.* Submit the query in file Q9.sql.

Q10: List the names of employee(s) assigned to the most projects. You can use joins in the inner and outer queries if you wish. You should have 2 rows. *Hint: aggregation is useful here as well.* Submit the query in file Q10.sql.

Part 3: Practice on IMDB

Now switch to your **imdb** database with the command `\c imdb`

You should have the database already created from the first assignment. If you need to get the data again, it is here: [IMDB dataset](#)

This dataset is much larger than the toy data you have been working with, so you will find that writing queries can be more complex, as you cannot as easily verify your work. The queries can also take longer to execute, especially if you make them more complex than necessary.

Start by recalling the schema with the `\dt` command. This dataset has the following tables:

ACTOR (id, fname, lname, gender)

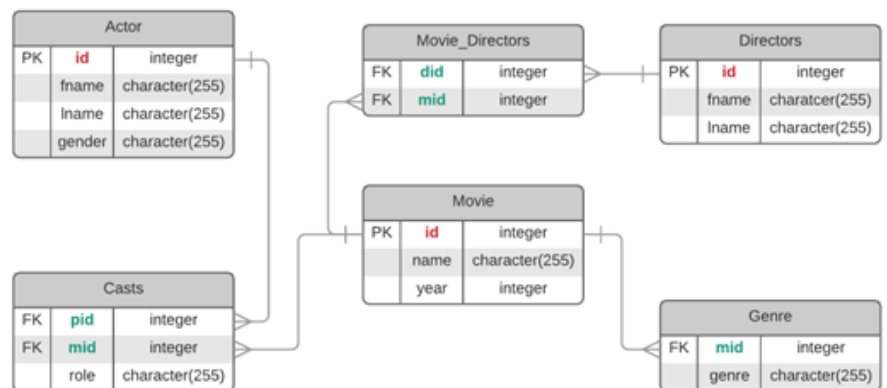
MOVIE (id, name, year)

DIRECTORS (id, fname, lname)

CASTS (pid, mid, role)

MOVIE_DIRECTORS (did, mid)

GENRE (mid, genre)



Q11: Compute how many movies were filmed in 1890. The output of your query should be a number. Submit the query in file Q11.sql.

Q12: For each year between 1890 and 1899 (including 1890 and 1899), compute how many movies were filmed that year. Your query should return the year in the first column and the appropriate number in the second column. Order the results in increasing order of the year. Submit the query in file Q12.sql.

Q13: List all directors who directed 30 or more 'Thriller' movies, in descending order of the number of thrillers they directed. Return the directors' first and last names and the number of thrillers each of them directed. Submit the query in file Q3.sql.

Q14: [Challenging query] We want to find actors who played exactly five distinct roles in the same movie during the year 1990. Write a query that returns the actors' first name, last name, and movie name. Submit the query in file Q14.sql.

If you finish early:

Get additional practice through fun SQL-based games.

Have you solved the SQL murder mystery yet? <https://mystery.knightlab.com/>

Also try out Lost at SQL: <https://lost-at-sql.therobinlord.com/>